

1 Requirements

In this section, I just want to explore what the primitives need to be able to do so that I approach their implementation in a structured way.

Ultimately, the primitive transformations modify the world in a specific way. They apply a transformation to the world state. In some cases that is a new participant and in other cases, it an existing participant that is modified. The point here is that the transformations are meaningful, they exist as part of a narrative.

1.1 Add

So lets start with the basic add primitive:

This transformation needs two participants, `addend1` and `addend2`. Both `addend1` and `addend2` must be numbers, they can be whole numbers, or floats. The resulting participant is a float or an int. So the meaning of the participants correctness is made even more clear and the invariants will be able to enforce the correct meaning.

So it is clear that I also need meaningful primitive types for the participants. The meaning of `addend1` and `addend2` are that they are part of a transformation but also must be of type float or int. So I need to establish that meaning and there is a mechanical sequence that can get its type to ensure that the participant has the correct meaning, the meaning of type is realized by that mechanical sequence.

So the transformation accepts two participants and generates a participant. The types of all the participants are known and the invariant is that both `addend1` and `addend2` have to be of type float or int. This is the meaning of the transformation and the conditions under which it will succeed.

Using this, the transformation can be invoked by a narrative. So I need to also provide the syntax for how it can be invoked by a narrative. For example, store result of add `addend1` and `addend2` in `result_name`. The narrative can be parsed and the actual participants can be identified using this.

So now I also set up parsing rules for the transformation. If I see the keyword `add`, then I know how to parse the rest of the line, so I can use the parser to access the structure from the primitive's meaning.

1.2 General Rule

In general, this process is assigning meaning to the primitives of a programming language and transforming programming into an act of constructing meaning. It is creating a semantic toolkit that consists of the meaningful primitives and using these primitives you can construct more meaning and it is computable. It is the meaning of anything mechanical, so the type of the participant has a specific meaning, float, int, boolean etc. So if the meaning of `add` is that it must have addends that are a float or an int, then the meaning can be enforced to ensure that the addends have the correct meaning.

Its actually very simple, this is just programming meaning. Its just hard to rewire the mind to think of programming this way because it is not doing anything new but it is just unambiguously constructing the meaning.

2 Addition

2.1 Multiplication

In this section, I want to explore how the add primitive can be used to build the meaning of multiplication. Multiplication is just repeated addition, so multiplication it is a great case study for how to build more meaning using narratives built from primitives.

However, clearly I can establish multiplication on it own as a primitive because its essentially a primitive operator and there are clearly many optimizations, it is just a very simple case study that shows how meaning can be built on top of primitives.

So despite me establishing the meaning of multiply as a primitive, if I were to proceed with designing multiplication using repeated addition, I would do it as follows:

Multiply multiplicand and multiplier.

Store 0 in result. Store 0 in counter. While the counter is less than multiplier, store result of add multiplicand and result in result then store result of add 1 and counter in counter.

It is clear from this that to create the design that realizes this narrative, you need to establish how while will be automatically translated into a design structure. In this case, it is simply a conditional check that counter is less than the multiplier and apply the subsequent store operations. This can be automatically constructed as a design from the narrative and then an engine can reason about it.

Note that I don't specify the invariants in the narrative, since the transformations are built from computable semantic primitives, the invariants are established through construction.

Note that the conditional check in the while loop isn't a mechanical, it is inherently meaningful. It is reasoning about the state of the world and making a decision about which behavior to exhibit. If it fails, the meaning of that failure is unambiguous.

2.2 Bill Total Example

Using the add primitive, you can establish the meaning of behaviors. For example consider the behavior below in a restaurant payment app which is called add bill.

Store 0 in bill-total. For each diner in the group, store the result of add diner-owes and bill-total in bill-total.

Now in a larger narrative you can use add bill

After the diners request the bill, store the result of add bill in total-owed.

Since add bill is built from primitives that have unambiguous invariants, the correctness of the narrative can be established. For example if the each diners owes's type is not a number, the engine can reason that the add will fail.